

AlphaBlokus: Curriculum-Driven Self-Play and Adaptations for Blokus

Alex Racapé
Columbia University
abr2184@columbia.edu

Abstract—Games have long served as testbeds for artificial intelligence, offering controlled environments for developing and evaluating learning algorithms. While AlphaZero’s combination of deep reinforcement learning, Monte Carlo Tree Search, and self-play has achieved remarkable success across two-player board games, its extension to complex multiplayer domains remains underexplored. This paper presents the first end-to-end adaptation of the AlphaZero framework to Blokus, a four-player, perfect-information placement game with a vast combinatorial action space. We introduce a scalable environment, a Transformer architecture, and a dimension-based curriculum learning strategy to address the challenges of the high-dimensional, multiplayer setting. Our empirical evaluation shows that the model significantly outperforms strong heuristic baselines with orders of magnitude fewer training games compared to prior AlphaZero implementations. Ablation studies explore the critical impact of architectural and hyperparameter choices on performance. These results build on prior research in self-play reinforcement learning, extending established methods to a multiplayer domain and providing empirical evidence for the effectiveness of curriculum learning and Transformer architectures in large action spaces.

Index Terms—Curriculum Learning, Reinforcement Learning, Monte Carlo Tree Search

I. INTRODUCTION

GAMES have been a proving ground for artificial intelligence research since the discipline’s inception. From early combinatorial search in checkers and chess to the high-dimensional sensory challenges of Atari, well-defined games provide a controlled environment where hypotheses can be simulated and tested at scale [1]. In this context, deep reinforcement learning (RL) coupled with Monte Carlo Tree Search (MCTS) and self-play has emerged as one of the most influential algorithmic approaches. The paradigm was first introduced by Google DeepMind with AlphaGo, which achieved superhuman performance in the game of Go [2]. The team iterated on this methodology in AlphaGo Zero and AlphaZero, which aimed to generalize the algorithm while eliminating the need for human data and domain-specific knowledge [3], [4].

Self-sufficiency is a fundamental advantage of this approach. The model bootstraps its own training data by iteratively playing against versions of itself, progressively generating better data without the need for external annotations. Consequently, this framework scales well in contexts where there is some reward criteria associated with a series of actions.

With the proliferation of large language models (LLMs), this technique is even being leveraged to improve reasoning capabilities post training [5]. Reinforcement learning is increasingly being used to train and fine-tune LLMs [6], and this learning framework can help models explore the most promising chains of thought.

While the AlphaZero framework has proven to be remarkably effective across multiple domains, it is not without significant limitations. One major challenge is sample efficiency. AlphaZero requires an enormous amount of self-play data, often generating millions of games to achieve expert-level performance. This requires substantial computational resources and time, making it impractical for many applications. Additionally, the framework’s success hinges on careful tuning of the neural network architecture, search parameters, and training schedule, which can be brittle and difficult to generalize across domains. Although there has been some promising work extending AlphaZero to multiplayer and non-zero-sum settings [7], these areas remain comparatively underexplored and present valuable opportunities for advancing the framework. Together, these limitations motivate research into improving sample efficiency, reducing computational cost, and broadening the framework’s applicability to more complex environments.

Blokus presents a particularly compelling testbed. It is a perfect-information, deterministic placement game for four players where each player attempts to maximize territorial coverage on a shared 20×20 grid using 21 polyomino pieces of varying size and shape. The branching factor is an order of magnitude larger than that of Go, and the multiplayer setting introduces added complexity.

In this paper we report the first end-to-end AlphaZero implementation for Blokus. Our study makes three primary contributions:

- **Proof-of-concept adaptation.** We design and implement a scalable Blokus environment, MCTS procedure, and Transformer-based neural architecture capable of handling the game’s high-dimensional placements and four-player turn order.
- **Dimension-based curriculum learning.** While other work has explored various curriculum approaches, this is the first work to combine a dimension-based curriculum with a vision Transformer architecture.
- **Empirical evaluation and analysis.** We provide quantitative evidence that the model surpasses strong heuristic baselines using orders of magnitude fewer training games

compared to similar approaches. Additionally, we present ablation studies that highlight the critical role of architectural choices and hyperparameter tuning within this framework.

By adapting the AlphaZero framework for Blokus, this research explores ways to extend self-play reinforcement learning to complex, multiplayer environments and offers insights into improving efficiency, scalability, and generalization in large-state-space games.

II. RELATED WORK

A. AlphaZero’s Progression

DeepMind’s AlphaGo demonstrated that a deep neural network combined with MCTS was capable of mastering Go by learning through self-play [2]. However, AlphaGo relied heavily on domain-specific knowledge and supervised learning from expert games. In contrast, AlphaGo Zero achieved super-human performance using only self-play, removing the need for human examples [3]. This approach was further generalized in AlphaZero, which minimized game-specific tuning and employed a single neural network architecture to achieve state-of-the-art performance in multiple games, including Go, chess, and shogi [4]. The central innovation across these systems is the integration of MCTS and reinforcement learning to guide decision-making in complex game trees. Intuitively, each game can be viewed as a tree of possible states and actions, where each node represents a board configuration and each branch corresponds to a legal move. The challenge is to identify which action to select at each decision point to maximize long-term success. To address this, AlphaZero employs a neural network that takes a board state as input and outputs two predictions: a policy, representing the relative quality of available moves, and a value, estimating the expected outcome from the current player’s perspective. MCTS complements this by simulating potential game trajectories, using the network’s policy and value estimates to focus on the most promising branches. There are two phases during training. During self-play, the agent plays games against itself using a guided MCTS. These games then provide training data to improve the network’s predictions. This combination of self-play and MCTS has become the foundation for modern board game AI.

1) *Multiplayer games and New Architectures:* Subsequent work has extended AlphaZero in several ways. Multiplayer AlphaZero modified the algorithm to handle games with more than two players [7]. The essential change was in the value head of the network where they output a value vector as opposed to a single scalar. With this simple modification, their networks were able to learn effective strategies in 3-player games, indicating that self-play MCTS can be adapted beyond two-player settings.

In parallel, researchers have explored alternative neural network architectures. The original AlphaZero used a ResNet architecture with a fixed board dimension. Other work has explored Graph Neural Networks (GNNs) and Vision Transformers (ViTs) [8] [9]. GNNs are scalable and allow for a flexible input size, and ViTs have recently achieved state-of-the-art performance in computer vision [10] [11]. Rather than

relying on convolutions, ViTs process the board as a sequence of patches using attention. The output representation is equal in dimension to the input, making it natural to accommodate different board sizes and even multiple games with one model. These results suggest that Transformers can generalize across board geometries. While ViTs lack the same inductive bias as CNNs, global attention and its flexibility may help the model to better learn long-range dependencies. Motivated by these advantages, this work adopts the Vision Transformer architecture to leverage its flexibility and strong performance in high-dimensional input domains.

B. Curriculum Learning

Curriculum learning is a technique used in RL to manage the complexity when learning a new task. Similar to the way humans learn, the idea is to train the model on a simpler task before scaling up the difficulty. This can take many forms, but it is built on the premise that features and strategies learned during one phase will transfer to the main task. Examples include changing the reward function, state space, or environment all together. In reinforcement learning, curriculum methods have been shown to improve sample efficiency and final performance.

One notable example in the board game sphere is the end-game-first curriculum, which initially trains on game positions near the end of a match [12]. With the end-game-first curriculum, the consequences of actions are clearer and the value signal is stronger. As training progresses, earlier and more complex game stages are gradually introduced. This approach focuses learning on high-value, informative experiences and prevents the agent from wasting early training on uninformed exploration.

Another approach was introduced in ScalableAlphaZero, which varies the board size throughout the curriculum. Their system begins by training on smaller board dimensions, then progressively scales to larger boards [8]. To enable this scalability, they rely on a GNN architecture. While GNNs offer flexibility, our work instead adopts the more modern Vision Transformer (ViT) architecture, using attention. While the AlphaViT paper did not implement a well defined curriculum, they trained versions of the model on multiple games at once, indicating a strong capacity for transfer learning [9]. In this work, we combine these ideas by using a ViT to support a curriculum over board size in Blokus, aiming to leverage both the architectural flexibility of Transformers and the training efficiency gains of curriculum learning.

C. Factored Action Spaces

Blokus provides a challenge for these algorithms due to the massive discrete action space. Each turn involves a combinatorial number of legal moves resulting from the number of pieces, the orientations, and the large board dimension. Prior work has tackled similar challenges in other board games by factoring actions into sequential sub-decisions within MCTS. One approach takes advantage of correlations across similar moves by grouping them together. By adding an extra layer of decision, they were able to drastically reduce the branching

factor of the search tree [13]. Similar work addressed the multi-step turns in Arimaa by modeling each atomic step as a separate node in the search tree, avoiding the complexity associated with enumerating all possible actions [13]. These approaches show that decomposing large, structured action spaces can make MCTS practical in complex domains.

III. ALPHABLOKUS

A. Game Logic

The core game engine manages board state and generates legal moves. Boards are represented with bit-packed integers, and the information for each space can be stored in a single byte. The lower four bits record which player occupies the square, while the upper four bits encode per-player restriction flags that prevent players from placing a tile adjacent to one of their own. For example, any space that is full starts with four ones then a bit for the occupying player. In addition to this bit mask, the engine maintains the set of corner squares that each player can build off of. In combination, these low-level optimizations help to quickly check the legality of any move. Another key implementation detail is that moves are defined as the placement of a single tile. This decomposes one large action into multiple constituent tile placements. To do this, the game logic simultaneously keeps track of the overarching possible piece placements along with the set of corresponding legal tiles. Notably, the player continues placing tiles until they run out of legal moves. The game and MCTS logic are implemented in Rust and exposed to Python through pyo3 bindings. The model architecture was built from scratch in Pytorch, and the source code is publicly available.

B. Monte Carlo Tree Search

Our Monte-Carlo Tree Search follows the same structure as AlphaZero (selection $\rightarrow$ expansion $\rightarrow$ evaluation $\rightarrow$ back-propagation).

Selection: From the root state we descend to a leaf by repeatedly choosing the action

$$a = \operatorname{argmax}_{a'} \left[\underbrace{\left(c_{\text{puct}} \cdot \frac{\sqrt{N_s}}{1 + N_{(s,a')}} \right) \cdot P_{(s,a')}}_{\text{exploration}} + \underbrace{V_{(s,a')}}_{\text{exploitation}} \right]$$

where V is the average value of a node, P is the neural-network prior, and N visit count. Dirichlet noise is mixed into P at the root to keep play diverse.

Expansion and Evaluation: When we reach a previously unvisited, node we create a branch for every legal action and query the network once to obtain the node values $\hat{v}$ and action priors P .

Backpropagation: The outcome v (terminal score or $\hat{v}$) is propagated up the path, incrementing visit counts and updating V with a running average. After a number of simulations (see Table I for hyper-parameters), we pick the root action with the highest visit count and reuse the resulting sub-tree for the next move, improving subsequent searches.

TABLE I
MCTS HYPERPARAMETERS

Parameter	Value
Simulations per Move	100
Sample Moves	30
c_{puct}	4
Dirichlet α	0.3
Exploration Fraction	0.2
Worker Processes	50
Games per Round	100

C. Architecture

We employ a Vision Transformer (ViT) encoder, customized for discrete board states [10]. Each cell of the Blokus board is mapped to a single token, yielding at most 400 input tokens per position. Learnable one-dimensional positional embeddings are added to the token sequence. The ViT authors reported that two-dimensional encoding schemes did not confer any measurable benefit since the model was able to learn this spatial information in the embeddings. Because our “patch” size (one cell) is comparable to the 16×16 patches in the original study, self-attention should recover this spatial structure without additional inductive bias. However, we looked to validate this assumption through an ablation in which the ViT encoder was replaced by a convolutional ResNet [14]. More details for the architecture can be found in Table II

TABLE II
MODEL ARCHITECTURE PARAMETERS

Parameter	Value
Maximum Board Dimension	20
Embedding Dimension	128
Number of Attention Heads	4
MLP Dimension	256
Number of Transformer Blocks	4
Dropout Rate	0.1

Analogous to BERT and ViT, a prepended [CLS] token learns a holistic representation that is used in the value head of the network. A separate policy head projects the final token embeddings to a distribution over legal actions. Both heads are trained jointly with the sum of their cross-entropy losses.

$$\mathcal{L} = - \underbrace{\sum_{i=1}^d \sum_{j=1}^d p_{i,j}^* \log p_{i,j}}_{\text{policy loss}} + - \underbrace{\sum_{k=1}^4 v_k^* \log v_k}_{\text{value loss}}$$

where p^* and v^* are the MCTS targets. The model takes any dxd input up to a maximum of 400 tokens in addition to the [CLS] token. Each token contains 5 values indicating which player occupies that space and whether it would be a legal placement. These channels are rotated to orient the state to the current player. The output is a policy over the dxd possible actions along with a four element vector with the win probability (value) for each player.

Self-play is parallelized across worker processes that queue batched inference requests to a shared GPU. After each round of K games, the Transformer is updated for T gradient steps

using this new data along with older games from a replay buffer. The hyperparameters for training are listed in Table III.

TABLE III
TRAINING PARAMETERS

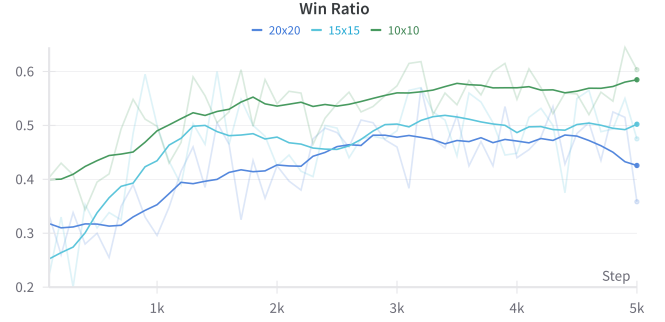
Parameter	Value
Learning Rate	0.01
Weight Decay	10^{-4}
Batch Size	512
Training Steps per Round	100
Buffer Capacity	50,000 positions

IV. EXPERIMENTS

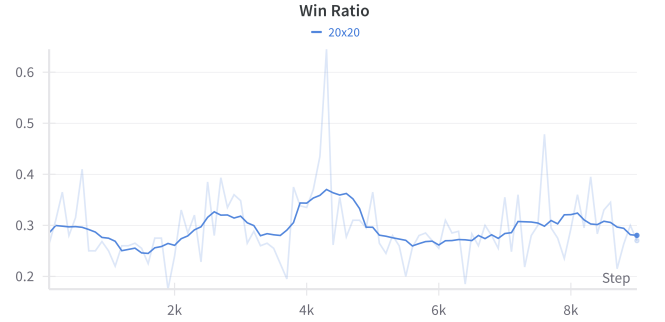
In our main experiment, we evaluate the effectiveness of the curriculum by comparing it against a model that is purely trained on a 20x20 board. The curriculum consisted of 50 rounds of training at 10x10, 15x15, then 20x20. With 100 games worth of data generated per round, our first model was trained on a total of 15,000 games of self-play. For reference, AlphaZero was trained on 44 million games in order to master chess [4]. Rounds at the lower dimensions took significantly less time, since the games end quicker with fewer simulated moves. When training purely at 20x20, 15,000 games was no longer feasible due to our resource constraints, so we trained on an equivalent number of total moves or 90 total rounds. With the same number of simulated moves and training steps, this also roughly approximates equal wall-time. The curriculum ran in total for 95 hours, while the baseline model spent 104 hours training only on the 20x20 board. It should be noted that the evaluation games are also part of this time, and they likely account for the difference in the two times. Self-play accounted for the vast majority of training time, and both models used the same hyperparameters and number of training steps.

We conducted two controlled ablations on the 10x10 board since it was the fastest configuration to evaluate. First we swapped the Vision Transformer for a ResNet architecture. This architecture was set up for a fixed dimension board. Second, we added data augmentation, step-decay learning-rate scheduling, and a greater number of training steps per round. With this we hoped to address the plateauing and poor sample efficiency in our previous rounds of training. The learning rate schedule featured a drop from .01 to .001 after 3000 steps since this is where we observed a plateau during our first round of training. For data augmentation we added random rotations which take advantage of the natural symmetries in the game. In conjunction, we increased the number of training steps per round to 400 in order to take advantage of this added sample efficiency.

At the end of each training round we gauged playing performance by having the current model play 100 games against three baseline “random-greedy” opponents. Notably, this baseline selects each tile uniformly at random but continues to extend that piece as much as is legally possible. This is due to the way our environment decomposes a piece into successive single-tile actions. For instance, if a four-tile polyomino can be fully played, the agent will not pass after



(a) Curriculum



(b) Baseline

Fig. 1. Training evaluation against quasi-random agents

two tiles, instead completing the entire four-tile trajectory before ending its turn. The policy is therefore locally random in its tile choices yet greedy over the course of one turn. While this agent is not completely random over the actions in Blokus, it yields a level of play that closely mirrors an untrained network in our framework.

V. RESULTS

Figure 1 compares the curriculum schedule with a baseline trained solely on the full 20x20 board. The curriculum yields consistently higher win ratios at every stage. The model surpasses 0.60 on the 10x10 board, stabilizes near 0.53 on 15x15, and still attains just under 0.50 after transferring to 20x20. Meanwhile, the baseline never rises above roughly 0.33. This suggests that mastering simpler environments first allows the network to accumulate useful priors that transfer upward in scale, effectively warming up the policy before confronting the full combinatorial complexity. The lack of progress in the baseline model likely indicates an issue with hyperparameters. In separate mistuned runs we observed the same flat learning curves. If the MCTS explores too much, the noisy training signal may prevent the model from converging (Figure 3). The curriculum’s continued progress under the same conditions points to greater robustness with respect to hyper-parameter selection. In all experiments, the win ratio statistic had surprisingly high variance. This is most likely due to the fact that these evaluations have three random opponents playing simultaneously. As opposed to two player games, it is possible in this case for a perfect model to lose if the three



Fig. 2. Policy and Value Losses for the Curriculum

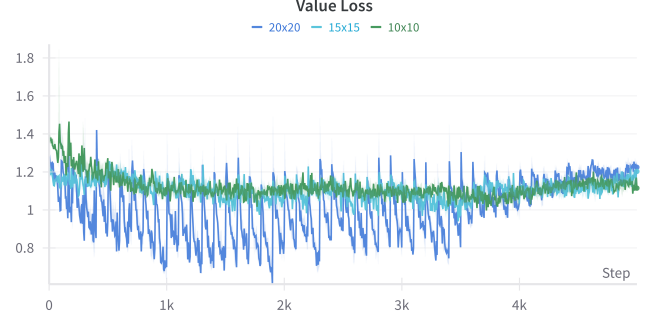
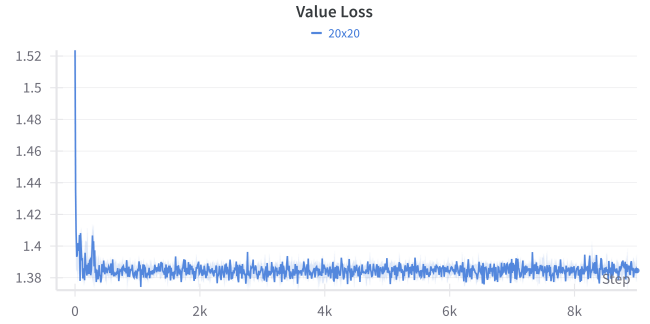


Fig. 3. Policy and Value Losses for the Baseline



random players happen to “gang up” and block the model. Due to the variance, we added the moving average over the past 10 rounds for clarity.

The loss functions do not change much over time for the curriculum either. This is to be expected since the model is constantly working with different data that reflect changing strategies, and other work has observed similar loss curves [7]. However, performance seems to plateau after roughly 2,000 training steps for each stage. Open-source AlphaZero implementations counteract such plateaus by lowering the learning rate. Our fixed rate is probably too high, as evidenced by the jagged value-loss trace on 20×20 (Figure 2). This observation motivated the learning-rate decay explored in our last ablation study.

All three curriculum curves begin above a uniform 0.25 win ratio. Blokus has a modest first-mover advantage [16], yet the advantage diminishes with larger boards, explaining the lower initial win rate on 20×20. The staggered policy losses in Figure 2 also highlight the inherent difficulty associated with the larger board dimension.

Figure 4 shows the results from our first ablation which utilized a ResNet architecture. The results were very similar to our baseline run for the 20×20 board size. The loss and win rate stagnates, likely due to the same issues discussed above.

Comparable plateaus emerged in the run that combined data augmentation with a stepped learning-rate schedule. However, it was able to make progress after roughly 9,000 training steps. Because of this initial stall, the model never quite matched

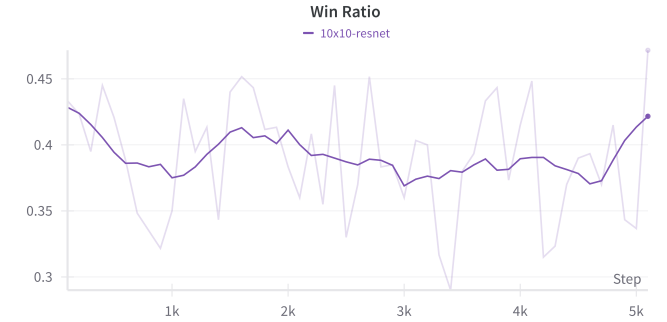


Fig. 4. Training evaluation for ResNet

the peak performance of our first curriculum experiment. While the changes in Figure 5 may look minor, the loss functions in Figure 8 depict a dramatic shift accompanying this improvement in win ratio.

Taken together, the experiments demonstrate that the system is able to learn meaningful policies rather than drifting in a uniform or purely random pattern. The staged curriculum consistently outperforms a direct 20×20 baseline, maintains transferable skill across board sizes, and remains productive even under sub-optimal hyper-parameters. Despite the brittleness with respect to learning-rate and exploration constants, our curriculum results exhibit a monotonic rise in win ratio and a policy loss that departs from the log-uniform baseline,

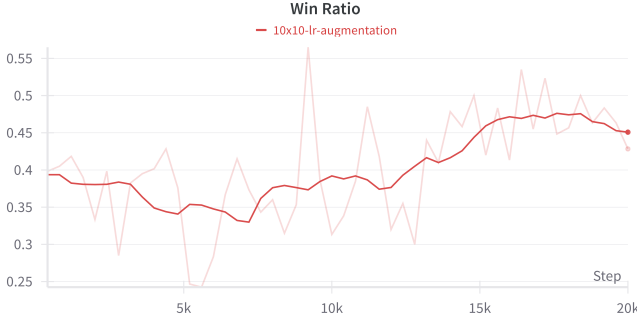


Fig. 5. Training evaluation with LR-scheduling and Data Augmentation

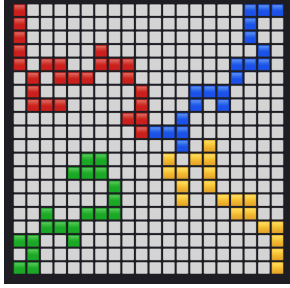


Fig. 6. Self-play opening observed after training

confirming that the model is capturing non-trivial aspects of Blokus play. Qualitative analysis also shows that the model learns human-like behavior such as racing towards the center of the board (Figure 6).

VI. CONCLUSION

This work delivers the first end-to-end demonstration that an AlphaZero, Transformer-based agent can learn to play the four-player game of Blokus. Factored actions and a dimension-based curriculum allowed the AlphaZero framework to be adapted to a game with novel complexity. With our approach we show evidence of progress and reasonable strategy after only 15,000 self-play games which is orders of magnitude fewer than comparable AlphaZero implementations. Despite the ablations shortcomings, the sensitivity to learning-rate and exploration constants exposes a lingering brittleness that merits future work. This project is a rare example of an extension to multiplayer, which incorporates a unique action decomposition scheme. While AlphaBlokus does not achieve state of the art mastery, it provides evidence that curriculum design, modern sequence models, and careful engineering can extend self-play reinforcement learning to more complex domains.

VII. FUTURE WORK

This project has sparked many ideas, and highlighted areas that could easily be improved in the future. The first immediate change would be tuning the hyperparameters. Reducing the exploration constants may address the brittleness highlighted in the experiments. Due to time constraints, we kept the same hyperparameters that worked in our initial curriculum, but this

could be improved going forward. Second, data augmentation and learning rate scheduling has a lot of promise, and we were unable to fully experiment with these options. Our hypothesis is that these additions would improve the model with the right hyperparameters. Another improvement would be to implement an Elo score and a richer evaluation scheme. Other similar implementations organize tournaments against various strategies and model checkpoints at the end of each training round. This could provide a clearer evaluation, and it may reduce the high variance we observed. In terms of the overall structure, implementing the backend libraries in Rust was a valuable learning experience, but it did not result in meaningful performance gains. The main bottleneck is processing the queued requests in the main thread. In order to do this in Python, the game states must be pickled and then transformed into tensors before being processed and sent back. This data processing overhead was likely the biggest oversight with this approach. There are Rust bindings for PyTorch and PyTorch libraries that mmap state tensors to reduce this overhead. In the end sticking with Python libraries, may have been easier and more performant. The last direction would be to scale things up. In comparison to the scale of AlphaZero, it is unreasonable to expect superhuman performance here, and it would be interesting to experiment with longer training runs.

REFERENCES

- [1] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [2] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016.
- [3] D. Silver *et al.*, “Mastering the game of Go without human knowledge,” *Nature*, vol. 550, pp. 354–359, Oct. 2017.
- [4] D. Silver *et al.*, “Mastering chess and shogi by self-play with a general reinforcement learning algorithm,” *arXiv preprint arXiv:1712.01815*, Dec. 2017.
- [5] K. Kumar, T. Ashraf, O. Thawakar, R. M. Anwer, H. Cholakkal, M. Shah, M.-H. Yang, P. H. S. Torr, F. S. Khan, and S. Khan, “LLM post-training: A deep dive into reasoning large language models,” *arXiv preprint arXiv:2502.21321*, Feb. 2025.
- [6] X. Zhou *et al.*, “DeepSeek-R1: Incentivizing reasoning capabilities in LLMs via reinforcement learning,” *arXiv preprint arXiv:2404.13729*, Apr. 2024.
- [7] N. Petosa and T. Balch, “Multiplayer AlphaZero,” *arXiv preprint arXiv:1910.13012*, Dec. 2019.
- [8] S. Ben-Assayag and R. El-Yaniv, “Train on small, play the large: Scaling up board games with AlphaZero and GNN,” *arXiv preprint arXiv:2107.08387*, Jul. 2021.
- [9] K. Fujita, “AlphaViT: A flexible game-playing AI for multiple games and variable board sizes,” *arXiv preprint arXiv:2408.13871*, Nov. 2024.
- [10] A. Dosovitskiy *et al.*, “An image is worth 16×16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, Oct. 2020.
- [11] Z. Liu *et al.*, “Swin transformer: Hierarchical vision transformer using shifted windows,” *arXiv preprint arXiv:2103.14030*, Mar. 2021.
- [12] J. West, F. Maire, C. Browne, and S. Denman, “Improved reinforcement learning with curriculum,” *arXiv preprint arXiv:1903.12328*, Jun. 2019.
- [13] C. B. Browne *et al.*, “A survey of Monte Carlo tree search methods,” *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, Mar. 2012.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.
- [15] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, Minneapolis, MN, USA, Jun. 2019, pp. 4171–4186.
- [16] C. Chao, “Blokus game solver,” B.S. thesis, California Polytechnic State Univ., San Luis Obispo, USA, Dec. 2018.

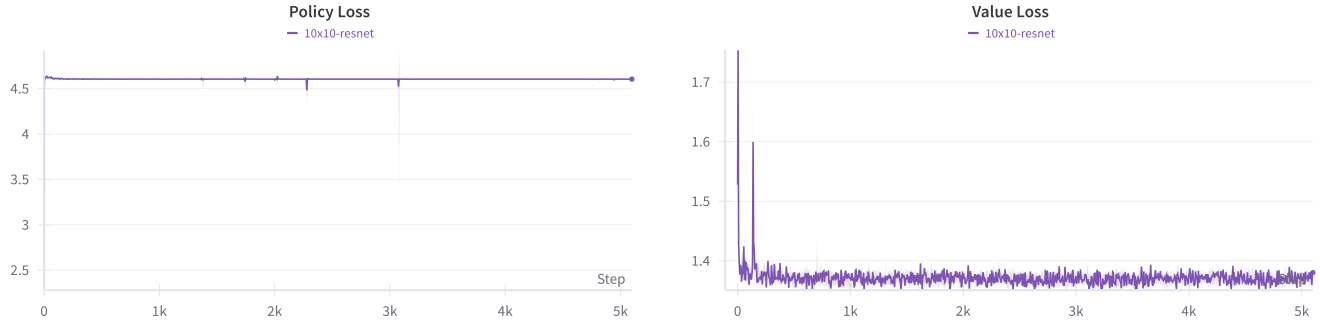


Fig. 7. Policy and Value Losses for the ResNet Ablation

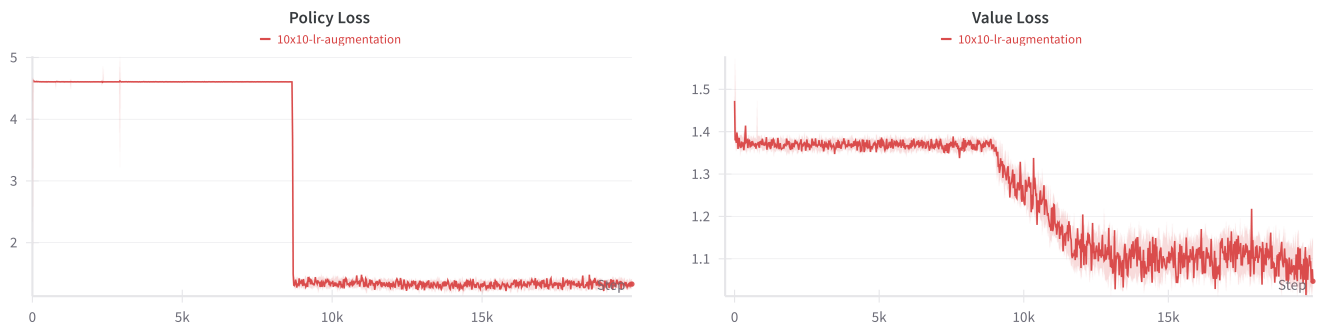


Fig. 8. Policy and Value Losses for the LR and Data Augmentation

APPENDIX A ABLATION LOSSES